Documentation

This document contains the documentation for the static class Pooler and the class Pool. It goes over every single available public method.

Navigate To Pooler

Navigate To Pool

Overview - Pooler

`Pooler` is a **static Class** that manages Unity object pools (both whole `GameObject`s and component types).
All operations are performed through static methods; you never instantiate `Pooler` yourself.

Methods

Category	What it does
GetObject	Retrieve a pooled instance (or a component from it) in a variety of ways – by prefab only, with a parent `Transform`, at a world-space position/rotation, or with custom positioning callbacks.
Release / PoolObject	Return an object to its pool (automatically deactivates it).
CreatePool	Explicitly create a pool for a prefab/component pair before first use, optionally pre-loading objects.
Count	Query how many objects exist / are active / are inactive for a given prefab (overall or per-type).
Utility	Miscellaneous helpers – clearing/destroying pools, setting “don’t destroy on load”, pre-loading, etc.

Category - GetObject

Pooler.GetObject

Declaration

```
public static GameObject GetObject(GameObject prefab, ... )
```

Declaration

```
public static GameObject GetObject(GameObject prefab, Transform parent, ... )
```

Declaration

```
public static GameObject GetObject(GameObject prefab, Transform parent,
bool instantiateInWorldSpace, ... )
```

Declaration

```
public static GameObject GetObject(GameObject prefab, Vector3 pos, Quaternion rot, ... )
```

Declaration

```
public static GameObject GetObject(GameObject prefab, Vector3 pos, Quaternion rot,
Transform parent, ... )
```

Shared Declaration

```
Action<GameObject> onNewInstance = null,
Action<GameObject> onGet = null,
Action<GameObject> onRelease = null,
Action<GameObject> onDestroy = null,
bool collectionCheck = true,
int defaultCapacity = 10,
int maxSize = 10000
```

Parameters

Parameter	Description
prefab	A prefab that you want to make a copy of.
pos	Position for the new object.
rot	Orientation of the new object.
parent	Parent that will be assigned to the new object.
instantiateInWorldSpace	When you assign a parent Object, pass true to position the new object directly in world space. Pass false to set the Object's position relative to its new parent.
onNewInstance	Action for when a new object is spawned
onGet	Action for when an object is retrieved from the pool
onRelease	Action for when an object is returned to the pool
onDestroy	Action for when excess object is destroyed
collectionCheck	True if checking for whether object has been pooled already
defaultCapacity	Initial memory allocation size for the internal stack
maxSize	Max size of the pool before extra objects are destroyed

Returns

GameObject The newly spawned object.

Description

Instantiates and clones the given prefab or retrieves from the pool and returns the **GameObject**.

Will automatically handle the active status of the **GameObject** and transform.

Has multiple overloads used to determine the initial spawning position and transform of the object.

Each overload contains the same shared extra declaration for more control. Assign actions to each stage of the pooled object life cycle from initial spawn, retrieval from pools, return to pool, and excess destruction.

Extra settings for the pool are also included for its size.

Pooler.GetObject<T>

Declaration

```
public static T GetObject<T>(GameObject prefab, ... ) where T : UnityEngine.Component
```

Declaration

```
public static T GetObject<T>(GameObject prefab, Transform parent, ... ) where T :
UnityEngine.Component
```

Declaration

```
public static T GetObject<T>(GameObject prefab, Transform parent,
bool instantiateInWorldSpace, ... ) where T : UnityEngine.Component
```

Declaration

```
public static T GetObject<T>(GameObject prefab, Vector3 pos, Quaternion rot, ... ) where
T : UnityEngine.Component
```

Declaration

```
public static T GetObject<T>(GameObject prefab, Vector3 pos, Quaternion rot, Transform
parent, ... ) where T : UnityEngine.Component
```

Shared Declaration

```
Action<T> onNewInstance = null,
Action<T> onGet = null,
Action<T> onRelease = null,
Action<T> onDestroy = null,
bool collectionCheck = true,
int defaultCapacity = 10,
int maxSize = 10000
```

Parameters

Go to table

Returns

T The component in the spawned object

Description

Instantiates and clones the given prefab or retrieves from the pool and returns the found **T** component in the object. Will log an error if no compatible component is found on the game object

Will automatically handle the active status of the **GameObject** and transform.

Has multiple overloads used to determine the initial spawning position and transform of the object.

Each overload contains the same shared extra declaration for more control. Assign actions to each stage of the pooled object life cycle from initial spawn, retrieval from pools, return to pool, and excess destruction.

Extra settings for the pool are also included for its size.

Pooler.GetObjects<T>

Declaration

```
public static IEnumerable<T> GetObject<T>(GameObject prefab, ... ) where T :
UnityEngine.Component
```

Declaration

```
public static IEnumerable<T> GetObject<T>(GameObject prefab, Transform parent, ... )
where T : UnityEngine.Component
```

Declaration

```
public static IEnumerable<T> GetObject<T>(GameObject prefab, Transform parent,
bool instantiateInWorldSpace, ... ) where T : UnityEngine.Component
```

Declaration

```
public static IEnumerable<T> GetObject<T>(GameObject prefab, Vector3 pos, Quaternion
rot, ... ) where T : UnityEngine.Component
```

Declaration

```
public static IEnumerable<T> GetObject<T>(GameObject prefab, Vector3 pos, Quaternion
rot, Transform parent, ... ) where T : UnityEngine.Component
```

Shared Declaration

```
Action<T> onNewInstance = null,
Action<T> onGet = null,
Action<T> onRelease = null,
Action<T> onDestroy = null,
bool collectionCheck = true,
int defaultCapacity = 10,
int maxSize = 10000
```

Parameters

Go to table

Returns

IEnumerable<T> An IEnumerable<T> list containing all found T in the spawned object, childrens included.

Description

Instantiates and clones the given prefab or retrieves from the pool and returns a IEnumerable list containing all found **T** components in the object and its children. Will log an error if no compatible component is found on the game object

Will automatically handle the active status of the **GameObject** and transform.

Has multiple overloads used to determine the initial spawning position and transform of the object.

Each overload contains the same shared extra declaration for more control. Assign actions to each stage of the pooled object life cycle from initial spawn, retrieval from pools, return to pool, and excess destruction.

Extra settings for the pool are also included for its size.

Category - Release / PoolObject

Pooler.PoolObject

Declaration

```
public static void PoolObject(GameObject instance)
```

Parameters

Parameter	Description
instance	The spawned object instance that you wish to pool

Description

Returns an active poolable object back to the pool. It can be called anywhere as long as the GameObject reference is given. You can call it via a manager that has a list of all objects, or from a script on the object itself.

Any object that isn't spawned via a pooler will throw an error if you attempt to Pool them. Pooling an object will

automatically deactivate the **GameObject**.

Prefabs designed for multipart pooling will deactivate the corresponding pass part as well as any further child object. Once all parts are pooled, the whole object will be returned back to the pool.

Category - CreatePool

Pooler.CreatePoolSingle

Declaration

```
public static Pool<GameObject> CreatePoolSingle(GameObject prefab, bool collectionCheck
= true, int defaultCapacity = 10, int maxSize = 10000)
```

Parameters

Parameter	Description
prefab	A prefab that you want to make a copy of.
collectionCheck	True if checking for whether object has been pooled already
defaultCapacity	Initial memory allocation size for the internal stack
maxSize	Max size of the pool before extra objects are destroyed

Returns

Pool<GameObject> Pool for GameObjects.

Description

Return a Pool which will store your prefab and returns the **GameObject** object to the user

Use this for when you wish to manually maintain your pools rather than rely on a static class or for when you wish to preload objects beforehand.

Pooler.CreatePoolSingle<T>

Declaration

```
public static Pool<T> CreatePoolSingle<T>(GameObject prefab, bool collectionCheck =
true, int defaultCapacity = 10, int maxSize = 10000) where T : UnityEngine.Component
```

Returns

Pool<T> Pool for T component.

Description

Return a Pool which will store your prefab and returns the **T component** object to the user

Use this for when you wish to manually maintain your pools rather than rely on a static class or for when you wish to preload objects beforehand.

Pooler.CreatePoolMulti<T>

Declaration

```
public static Pool<T> CreatePoolMulti<T>(GameObject prefab, bool collectionCheck = true,
int defaultCapacity = 10, int maxSize = 10000) where T : UnityEngine.Component
```

Returns

Pool<T> Pool for T component.

Description

Return a Pool which will store your prefab and return the list of **T component** objects to the user.

Use this when your prefab object is designed to have child objects be pooled.

Use this for when you wish to manually maintain your pools rather than rely on a static class or for when you wish to preload objects beforehand.

Category - Count

All count methods return **zero** and log a warning if the prefab has never been pooled.

Method Group	Description
Count Active	Counts all active objects (those not currently pooled)
Count All	Counts all objects (both pooled and not pooled objects)
Count Inactive	Counts all Inactive objects (those that are currently pooled)

Pooler.CountActiveOfAll

Declaration

```
public static int CountActiveOfAll(GameObject prefab)
```

Returns

Int The count of all active objects of the given prefab

Description

Return the count of all active objects that are of the type prefab. As you can create multiple different pools of the same prefab but with a different stored targeted component, this method allows you to track all active clones of the prefab.

Pooler.CountActive

Declaration

```
public static int CountActive(GameObject prefab)
```

Returns

Int The count of all active GameObject pool objects of the prefab

Description

Return the count of all active GameObject pool objects of the type prefab. As you can create multiple different pools of the same prefab but with a different stored targeted component, this method only counts objects spawned by the GameObject Pool.

Pooler.CountActive<T>

Declaration

```
public static int CountActive<T>(GameObject prefab) where T : UnityEngine.Component
```

Returns

Int The count of all active T component pool objects of the prefab

Description

Return the count of all active **T component** pool objects of the type prefab. As you can create multiple different pools of the same prefab but with a different stored targeted component, this method only counts objects spawned by the T component type Pool.

Pooler.CountAllOfAll

Declaration

```
public static int CountAllOfAll(GameObject prefab)
```

Returns

Int The count of all objects of the given prefab

Description

Return the count of all objects, both active and inactive, that are of the type prefab. As you can create multiple different pools of the same prefab but with a different stored targeted component, this method allows you to track all active clones of the prefab.

Pooler.CountAll

Declaration

```
public static int CountAll(GameObject prefab)
```

Returns

Int The count of all GameObject pool objects of the prefab

Description

Return the count of all GameObject pool objects, both active and inactive, of the type prefab. As you can create multiple different pools of the same prefab but with a different stored targeted component, this method only counts objects spawned by the GameObject Pool.

Pooler.CountAll<T>

Declaration

```
public static int CountAll<T>(GameObject prefab) where T : UnityEngine.Component
```

Returns

Int The count of all T component pool objects of the prefab

Description

Return the count of all **T component** pool objects, both active and inactive, of the type prefab. As you can create multiple different pools of the same prefab but with a different stored targeted component, this method only counts objects spawned by the T component type Pool.

Pooler.CountInactiveOfAll

Declaration

```
public static int CountInactiveOfAll(GameObject prefab)
```

Returns

Int The count of all Inactive objects of the given prefab

Description

Return the count of all Inactive objects that are of the type prefab. As you can create multiple different pools of the same prefab but with a different stored targeted component, this method allows you to track all active clones of the prefab.

Pooler.CountInactive

Declaration

```
public static int CountInactive(GameObject prefab)
```

Returns

Int The count of all inactive GameObject pool objects of the prefab

Description

Return the count of all Inactive GameObject pool objects of the type prefab. As you can create multiple different pools of the same prefab but with a different stored targeted component, this method only counts objects spawned by the GameObject Pool.

Pooler.CountInactive<T>

Declaration

```
public static int CountAll<T>(GameObject prefab) where T : UnityEngine.Component
```

Returns

Int The count of all inactive T component pool objects of the prefab

Description

Return the count of all inactive **T component** pool objects of the type prefab. As you can create multiple different pools of the same prefab but with a different stored targeted component, this method only counts objects spawned by the T component type Pool.

Category - Utility & Maintenance

Method Group	Description
Clear	Destroys all pooled object in target pool
Destroy	Destroys every object in the pool and the pool itself
DontDestroyOnLoad	Manages pools dontDestroyOnLoad status
Preload	Generates a given amount of inactive object for the desired pool

Pooler.ClearAllObject

Declaration

```
public static bool ClearAllObject(GameObject prefab)
```

Returns

Bool Whether the clear was successful or not.

Description

Clears all pools that use the given prefab regardless of the pool's return type. Will fail if the given GameObject isn't a prefab or if the prefab hasn't been used in any pools yet.

Successful clears will invoke the OnDestroy action assigned to each individual pool on each cleared object.

Does not clear any active objects, they will still be able to be returned back into the pool.

Pooler.ClearObject

Declaration

```
public static bool ClearObject(GameObject prefab)
```

Returns

Bool Whether the clear was successful or not.

Description

Clears the **GameObject** returning pool that uses the given prefab . Will fail if the given GameObject isn't a prefab or if the prefab hasn't been used in any pools yet.

Successful clears will invoke the OnDestroy action assigned to the pool on each cleared object.

Does not clear any active objects, they will still be able to be returned back into the pool.

Pooler.ClearObject<T>

Declaration

```
public static bool ClearObject<T>(GameObject prefab) where T : UnityEngine.Component
```

Returns

Bool Whether the clear was successful or not.

Description

Clears the **T component** returning pool that uses the given prefab . Will fail if the given GameObject isn't a prefab or if the prefab hasn't been used in any pools yet.

Successful clears will invoke the OnDestroy action assigned to the pool on each cleared object.

Does not clear any active objects, they will still be able to be returned back into the pool.

Pooler.DestroyPool

Declaration

```
public static bool DestroyPool(GameObject prefab)
```

Returns

Bool Whether the Destruction was successful or not.

Declaration

```
public static void DestroyPool(ref Pool<GameObject> pool)
```

Parameters

Parameter	Description
prefab	Prefab GameObject
pool	Local pool reference

Description

Destroys the **GameObject** returning pool that uses the given prefab. Will not invoke any OnDestroy actions set on the detected pool.

Will return false for invalid GameObject or no valid pool was found.

Does not clear any active objects, they will still be able to be returned back into the pool.

Ensure all poolable object are pooled before destruction or the pool is recreated again before pooling attempts

Pooler.DestroyPool<T>

Declaration

```
public static bool DestroyPool<T>(GameObject prefab) where T : UnityEngine.Component
```

Returns

Bool Whether the Destruction was successful or not.

Declaration

```
public static void DestroyPool<T>(ref Pool<T> pool) where T : UnityEngine.Component
```

Parameters

Parameter	Description
prefab	Prefab GameObject
pool	Local pool reference

Description

Destroys the **T component** returning pool that uses the given prefab. Will not invoke any OnDestroy actions set on the detected pool.

Will return false for invalid GameObject or no valid pool was found.

Does not clear any active objects, they will still be able to be returned back into the pool.

Ensure all poolable object are pooled before destruction or the pool is recreated again before pooling attempts

Pooler.SetPoolDontDestroyOnLoad / <T>

Declaration

```
public static void SetPoolDontDestroyOnLoad(Pool<GameObject> pool)
```

Declaration

```
public static void SetPoolDontDestroyOnLoad<T>(Pool<T> pool) where T :
UnityEngine.Component
```

Parameters

Parameter	Description
pool	Local pool reference

Description

Converts a pool into a DontDestroyOnLoad pool. All object transforms will be automatically set upon pool.

Pooled objects **won't** get destroyed upon scene changing.

Pooler.RemoveDontDestroyOnLoad / <T>

Declaration

```
public static void RemoveDontDestroyOnLoad(Pool<GameObject> pool)
```

Declaration

```
public static void RemoveDontDestroyOnLoad<T>(Pool<T> pool) where T :
UnityEngine.Component
```

Parameters

Parameter	Description
pool	Local pool reference

Description

Reverts a pool from the DontDestroyOnLoad pool back into a local pool. All object transforms will be automatically set upon pool.

Pooled objects **will** get destroyed upon scene changing.

Does nothing if the pool is already a local pool.

Pooler.PreloadObject / <T>

Declaration

```
public static void PreloadObject(Pool<GameObject> pool, int amount, Action<GameObject>
onNewInstance = null)
```

Declaration

```
public static void PreloadObject<T>(Pool<T> pool, int amount, Action<T> onNewInstance =
null) where T : UnityEngine.Component
```

Parameters

Parameter	Description
pool	Local pool reference
amount	Amount of new objects to preload
onNewInstance	Action that should run when for each newly created object

Description

Preloads a pool instance with the given amount. Will destroy the spawned object if it goes over the maximum allowed object count for the pool.

You can assign an action that triggers on each newly spawned object instance.

Preloading functions have a **Bug** on unity's end when the user has domain reload turned off. It will not function as intended in start, awake and onEnable calls. It will function properly in the final build, when domain reloading is on, or after the initial start, awake and onEnable calls. There is an error message that will warn the user about the bug if they have domain reloading turned off which can be disabled in the settings. Go to the toolbar Tools -> Sezylrin -> SimplePooling and check the checkmark to turn off the warning message.

Overview - Pool<T>

`Pool<T>` represents a **single object pool** (either *single* – one root object, or *multi* – a root with many child components).
It is created internally by `Pooler`; you normally interact with it through the static `Pooler` helpers, but the public members below can be called directly if you keep a reference to a `Pool<T>` instance.

Each pool stores one specified type be it a GameObject or a user defined component class (monobehaviour extensions) unless it is a multipool which doesn't allow for GameObject pooling

Properties

Category	What it does
Count	Public properties counting the amount of objects in the pool

Methods

Category	What it does
GetObject	Retrieve a pooled instance (or a component from it) in a variety of ways from the pool.
Release	No public release method – releasing is performed via `Pooler.PoolObject`
CreatePool	Pools are created by `Pooler.CreatePoolSingle` / `CreatePoolMulti`; the class itself does not expose a creator.
Callback Configuration	Set global callback actions for the pool objects life cycle
Pool Management	Miscellaneous methods for managing the pool

Count Properties

Property	Type	Description
CountAll	int	Returns the total number of objects associated with the pool, including both active and inactive instances.
CountInactive	int	Returns the number of inactive (pooled but not currently in use) objects in the pool.
CountActive	int	Returns the number of active objects currently in use.

Category - GetObject

Pool<T>.GetObject

Declaration:

```
public T GetObject()
```

Description:
Retrieves a single object from the pool and invokes the **`onGet`** action. If the pool is multi-type, returns the first detected component.

Returns:
A single instance of type **T**.

Declaration:

```
public T GetObject(Transform transform)
```

Description:
Retrieves a single object and sets its parent to the specified **`Transform`**.

Parameters:

transform	The parent transform to assign.

Returns:
A single instance of type **T**.

Declaration

```
public T GetObject(Transform transform, bool instantiateInWorldSpace)
```

Description:
Retrieves a single object, sets its parent, and optionally instantiates it in world space.

Parameters:

transform	The parent transform
instantiateInWorldSpace	If true, positions the object in world space; otherwise, relative to the parent.

Returns:
A single instance of type **T.**

Declaration:

```
public T GetObject(Vector3 position, Quaternion rotation)
```

Description:
Retrieves a single object and sets its position and rotation.

Parameters:

position	World-space position
rotation	World-space rotation

Returns:
A single instance of type **T**.

Declaration:

```
public T GetObject(Vector3 position, Quaternion rotation, Transform transform)
```

Description:
Retrieves a single object, sets its position and rotation, and assigns a parent transform.

Parameters:

position	World-space position
rotation	World-space rotation

transform	Parent transform

Returns:
A single instance of type **T.**

Declaration:

```
public IEnumerable<T> GetObjects()
```

Description:
Retrieves all active objects from the pool. For single pools, returns an **IEnumerable** with one item; for multi pools, returns all active components.

Returns:
An **IEnumerable<T>** of all stored components.

Declaration:

```
public IEnumerable<T> GetObjects(Transform transform)
```

Description:
Retrieves all active objects and sets the root object's parent to the specified **Transform**.

Parameters:

transform	The parent transform to assign.

Returns:
An **IEnumerable<T>** of all stored components.

Declaration:

```
public IEnumerable<T> GetObjects(Transform transform, bool
instantiateInWorldSpace)
```

Description:
Retrieves all active objects, sets the root object's parent, and optionally instantiates them in world space.

Parameters:

transform	The parent transform

instantiateInWorldSpace	If true, positions the object in world space; otherwise, relative to the parent.

Returns:
An **IEnumerable<T>** of all stored components.

Declaration:

```
public IEnumerable<T> GetObjects(Vector3 position, Quaternion rotation)
```

Description:
Retrieves all active objects and sets the root object's position and rotation.

Parameters:

position	World-space position
rotation	World-space rotation

Returns:
An **IEnumerable<T>** of all stored components.

Declaration:

```
public IEnumerable<T> GetObjects(Vector3 position, Quaternion rotation,
Transform transform)
```

Description:
Retrieves all active objects, sets the root object's position and rotation, and assigns a parent transform.

Parameters:

position	World-space position
rotation	World-space rotation
transform	Parent transform

Returns:
An **IEnumerable<T>** of all stored components.

Category - Release

Pool<T> does **not expose a public release method**.

Use this method instead

Category - CreatePool

Pool instances are **constructed internally** by **Pooler**.
There is no public constructor you need to call; you obtain a **`Pool<T>`** reference from here

Category - Callback Configuration

Pool instances can have their callbacks for each stage of the pooled object life cycle changed at any time

Pool<T>.SetOnNewInstance

Declaration:

```
public void SetOnNewInstance(Action<T> onNewInstance)
```

Description:
Sets the action to be executed when a new object is created for the first time.

Parameters:

onNewInstance	Action to invoke on new object creation.

Pool<T>.SetOnGet

Declaration:

```
public void SetOnGet(Action<T> onGet)
```

Description:
Sets the action to be executed when an object is retrieved from the pool.

Parameters:

onGet	Action to invoke on object retrieval.

Pool<T>.SetOnRelease

Declaration:

```
public void SetOnRelease(Action<T> onRelease)
```

Description:
Sets the action to be executed when an object is returned to the pool.

Parameters:

onRelease	Action to invoke on object release.

Pool<T>.SetOnDestroy

Declaration:

```
public void SetOnDestroy(Action<T> onDestroy)
```

Description:
Sets the action to be executed when an object is destroyed (e.g., when exceeding max pool size or during scene changes).

Parameters:

onDestroy	Action to invoke on object destruction.

Category - Pool Management

Pool<T>.Clear

Declaration:

```
public void Clear()
```

Description:
Destroys all pooled (inactive) objects in the pool and invokes the `onDestroy` action for each. Does not affect active objects.

Pool<T>.PreloadObjects

Declaration:

```
public void PreloadObject(int amount, Action<T> onNewInstance = null)
```

Description:
Pre-initializes a specified number of objects in the pool. If `onNewInstance` is provided, it overrides the existing callback for these objects.

Parameters:

amount	Number of objects to preload.
onNewInstance	Optional action to run for each new object.

Note:
If the number of preloaded objects exceeds the pool's `maxSize`, excess objects are destroyed.

Setting an onNewInstance will set the global onNewInstance for the pool. GetObject calls that spawn a new Instance will trigger the action set here.